

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(УНИВЕРСИТЕТ ИТМО)

Факультет «Систем управления и робототехники»

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине «Частотные методы»
на тему: «Жесткая фильтрация»

Студент:
Охрименко Ева

Преподаватели:
Догадин Егор Витальевич
Пашенко Артем Витальевич

г. Санкт-Петербург
2025

Содержание

1	Task. Жесткие фильтры	2
1.1	Краткое условие	2
1.2	Убираем высокие частоты	2
1.2.1	Предподготовка	2
1.2.2	Фиксирую b	4
1.2.3	Вывод	6
1.2.4	Фиксирую ν_0	7
1.2.5	Вывод	8
1.3	Убираем специфические частоты	9
1.3.1	Предподготовка	9
1.3.2	Случай, когда $b = 0$	9
1.3.3	Остальные случаи	11
1.3.4	Выводы по пунктам 1.3.2 и 1.3.3	12
1.4	Убираем низкие частоты?	13
1.4.1	Предподготовка	13
1.4.2	Графики	13
1.4.3	Выводы	16
2	Task. Фильтрация звука	17
2.1	Краткое условие	17
2.2	График исходного сигнала	17
3	Выводы	18

1 Task. Жесткие фильтры

1.1 Краткое условие

Рассмотрите функцию $g(t)$, заданную как:

$$g(t) = \begin{cases} a, & t \in [t_1, t_2], \\ 0, & t \notin [t_1, t_2], \end{cases}$$

и её зашумлённую версию:

$$u(t) = g(t) + b\xi(t) + c \sin(dt),$$

где $\xi(t) \sim U[-1, 1]$ — белый шум, а b, c, d — параметры.

- При $c = 0$ найдите Фурье-образ $u(t)$, обнулите его вне $[-\nu_0, \nu_0]$ и выполните обратное преобразование. Исследуйте влияние ν_0 и b .
- При ненулевых b, c, d обнулите Фурье-образ на выбранных частотах, подавляя шум и гармонику. Исследуйте влияние параметров.
- бнулите Фурье-образ в окрестности $\nu = 0$, пропустите сигнал через фильтр и оцените результат.

Ожидаемые результаты:

Графики исходного, зашумлённого и фильтрованного сигналов, а также их Фурье-образов. Выводы по каждому пункту.

1.2 Убираем высокие частоты

1.2.1 Предподготовка

Для начала выберу все нужные параметры для этого задания:

$$a = 4, t_0 = 0, t_1 = 3, c = 0, d = 5, b = 0.5$$

Тогда у меня получится прямоугольная функция:

$$g(t) = \begin{cases} 4, & t \in [0, 3], \\ 0, & t \notin [0, 3], \end{cases}$$

Теперь посмотрим, какая функция белого шума получилась:

$$u(t) = g(t) + 0.5\xi(t) + 0 \sin(dt),$$

где $\xi(t) \sim U[-1, 1]$ — равномерное распределение на интервале $[-1, 1]$.

Слагаемое с синусом отсутствует, поэтому колебаний у шума также не будет. Приведу код для задания функций и их отрисовки:

```
1 a = 4;  
2 t1 = 0;  
3 t2 = 3;  
4 c = 0;  
5 d = 5;  
6 b = 0.5;
```

```

8 g[t_] := Piecewise[{{a, t1 <= t <= t2}}, 0];
9 u[t_] := g[t] + b RandomReal[{-1, 1}] + c Sin[d t]
10
11 Plot[{g[t], u[t]}, {t, -1, 4},
12   Exclusions -> None,
13   AspectRatio -> 0.2,
14   PlotStyle -> {{Red, Thick}, {Gray, Thickness[0.001]}},
15   PlotPoints -> 9,
16   AxesLabel -> {t, "Amplitude"},
17   PlotLegends -> Placed[{"Original signal", "Noisy signal"}, Right],
18   ImageSize -> Large
19 ]

```

Листинг 1: Инициализация

Теперь посмотрим на график функций $g(t)$ и $u(t)$:

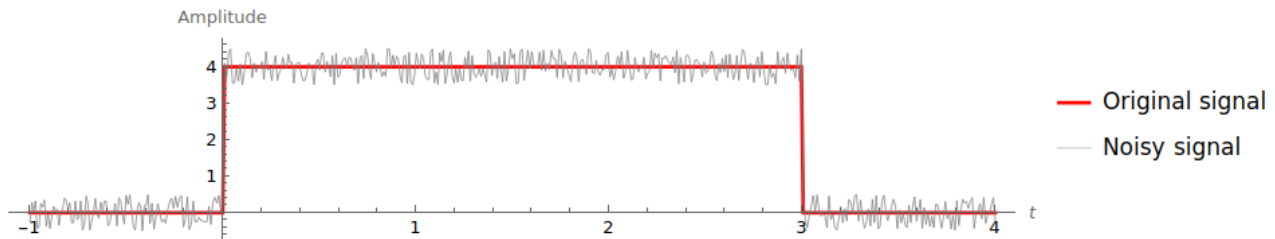


Рис. 1: График $g(t)$ и $u(t)$

Дальше в задании нужно найти фурье-образ $u(t)$, обнулить его значение на диапазоне $[-\nu_0, \nu_0]$ и восстановить сигнал с помощью обратного преобразования фурье.

Напишу функцию для такого фильтра. Передам в нее зашумненный сигнал, время записи сигнала, частоту частот дискретизации `sampleRate`. В самой функции я:

- Сделаю сигнал дискретным
- Сделаю дискретное преобразование Фурье
- Построю частотную ось
- Отфильтрую
- Сделаю обратное преобразование Фурье

```

1 fourierFilter[u_, {t_, tmin_, tmax_}, sampleRate_, v0_] :=
2 Module[{timeValues, signalValues, fftValues, freqRange, freqStep,
3   freqValues, filteredFFT, filteredSignal},
4 timeValues = Subdivide[tmin, tmax, (tmax - tmin)*sampleRate - 1];
5 signalValues = u /@ timeValues;
6 fftValues = Fourier[signalValues, FourierParameters -> {-1, 1}];
7 freqRange = Length[fftValues];
8 freqStep = sampleRate/freqRange;
9 freqValues = Table[(n - 1)*sampleRate/freqRange, {n, freqRange}];
10 freqValues = If[# > sampleRate/2, # - sampleRate, #] & /@ freqValues;
11 filteredFFT =
12 MapThread[If[Abs[#2] <= v0, #1, 0] &, {fftValues, freqValues}];
13 filteredSignal =
14 InverseFourier[filteredFFT, FourierParameters -> {-1, 1}];
15 {timeValues, Re[filteredSignal]}

```

Листинг 2: Функция фильтрации частот вне диапазона

1.2.2 Фиксирую b

Для начала выберем значения $\nu_0 = 0.5$ и $b = 0.5$. Теперь посмотрим на график получившегося отфильтрованного сигнала при этих значениях. Также я приведу графики модулей фурье образов сигнала $g(t)$, зашумленного сигнала $u(t)$ и отфильтрованного сигнала.

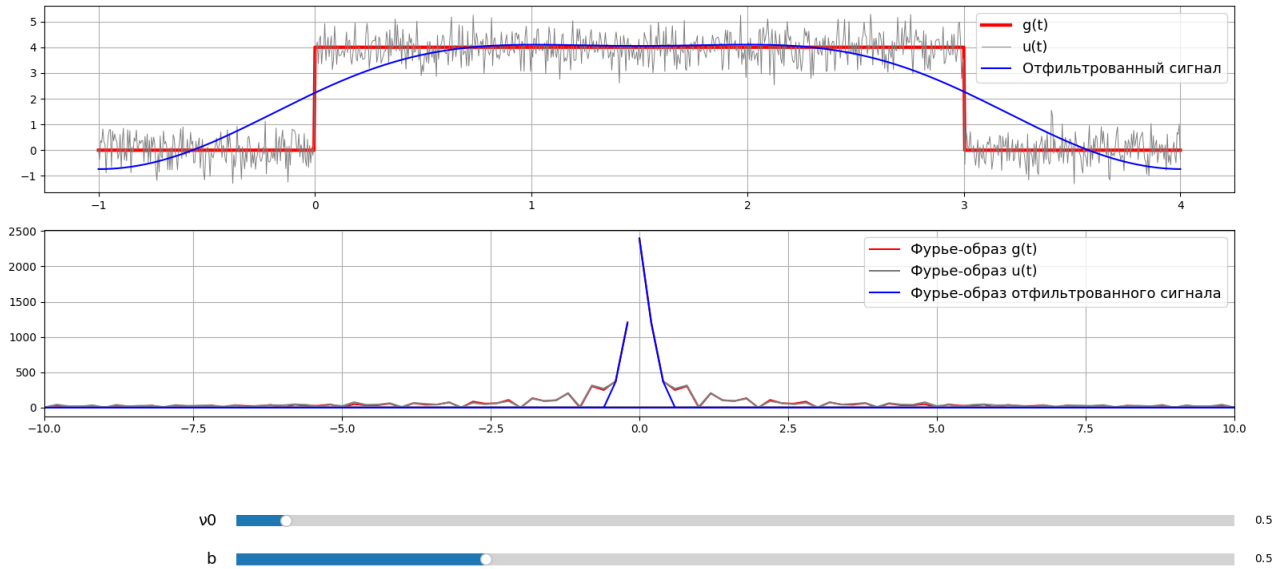


Рис. 2: Графики при $\nu_0 = 0.5$ и $b = 0.5$

Внизу можно заметить 2 бегунка, с помощью которых можно менять параметры. В этой части задания я зафиксирую параметр $b = 0.5$ и буду исследовать влияние на поведение функций параметра ν_0 . Выберу несколько $\nu_0 = \{1, 1.5, 3, 10\}$ и отрисую графики:

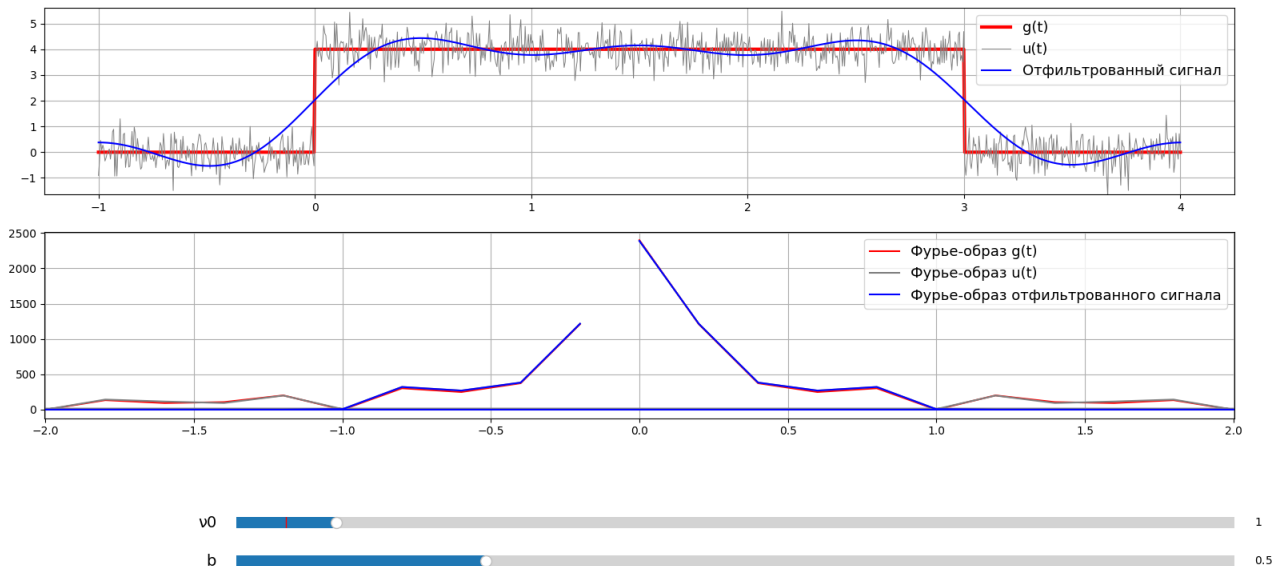


Рис. 3: Графики при $\nu_0 = 1$ и $b = 0.5$

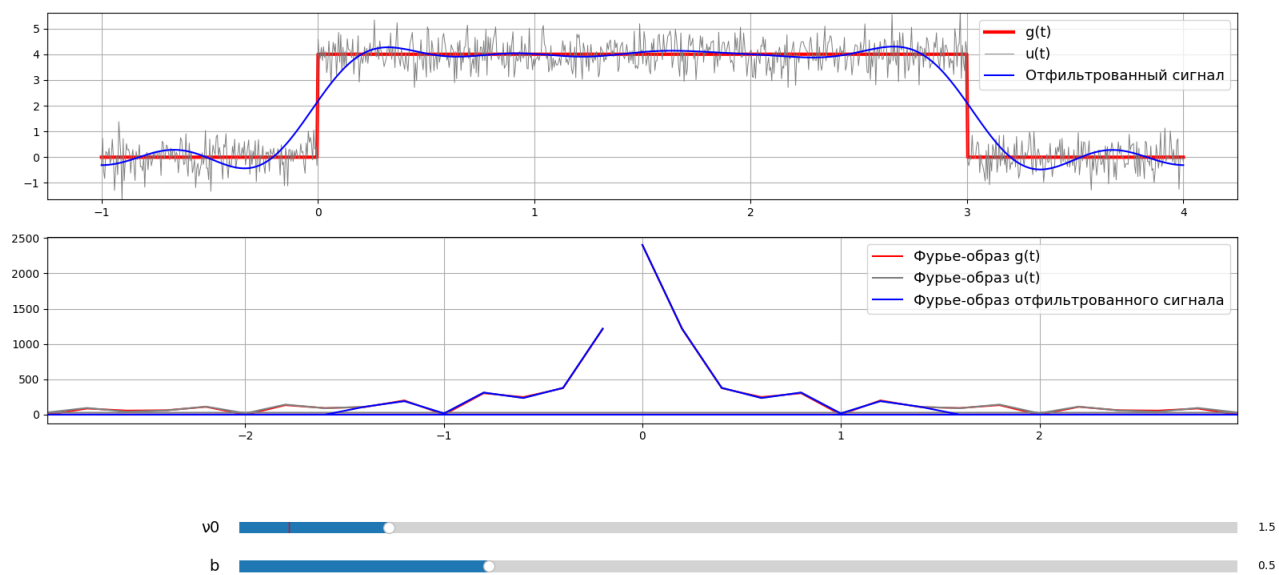


Рис. 4: Графики при $\nu_0 = 1.5$ и $b = 0.5$

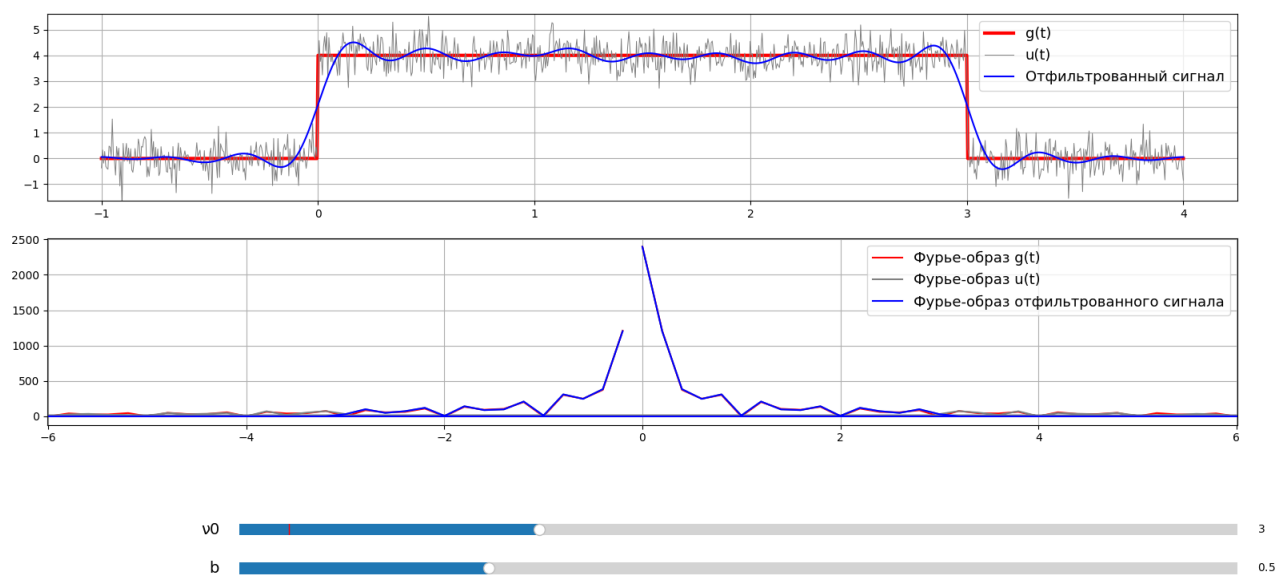


Рис. 5: Графики при $\nu_0 = 3$ и $b = 0.5$

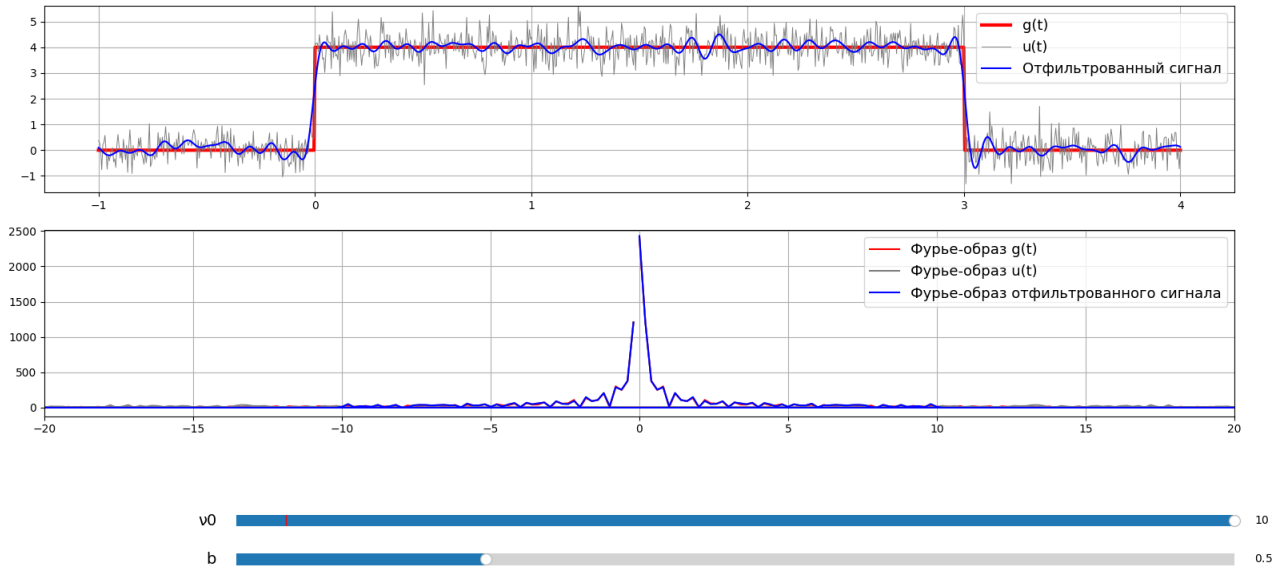


Рис. 6: Графики при $\nu_0 = 10$ и $b = 0.5$

1.2.3 Вывод

- С увеличением ν_0 изменялось количество колебаний отфильтрованного сигнала. Число гармоник увеличивалось, однако при большем значении ν_0 не всегда удавалось получить хорошо отфильтрованный сигнал. Наиболее удачным оказался график при параметрах $\nu_0 = 3$ и $b = 0.5$. На этом графике форма отфильтрованного сигнала практически идеально совпадает с исходной, а шум удалён наиболее эффективно.
- Можно заметить, что графики модулей Фурье-образов отфильтрованного сигнала и шума совпадают при всех выбранных значениях ν_0 . Теперь обратим внимание на синюю линию — модуль Фурье-образа. С увеличением ν_0 синий график постепенно начинает совпадать с остальными, практически полностью повторяя их форму. Это означает, что при увеличении частоты среза ν_0 фильтр пропускает больше частот, что приводит к лучшему сохранению формы сигнала.

1.2.4 Фиксирую ν_0

Для этого задания выберу несколько значений $b = \{0, 0.5, 1, 2\}$, чтобы исследовать поведение графиков при фиксированном значении $\nu_0 = 3$. Это значение было выбрано, поскольку ранее мне показалось, что при этом значении сигнал хорошо фильтруется. Ниже рассмотрим эти графики:

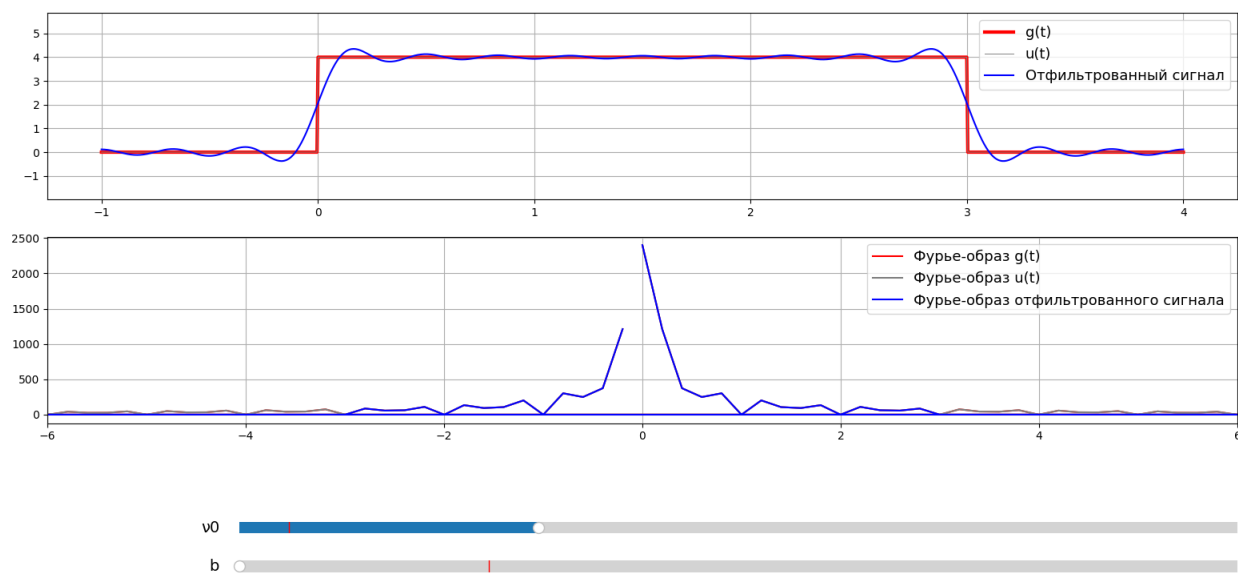


Рис. 7: Графики при $\nu_0 = 3$ и $b = 0$

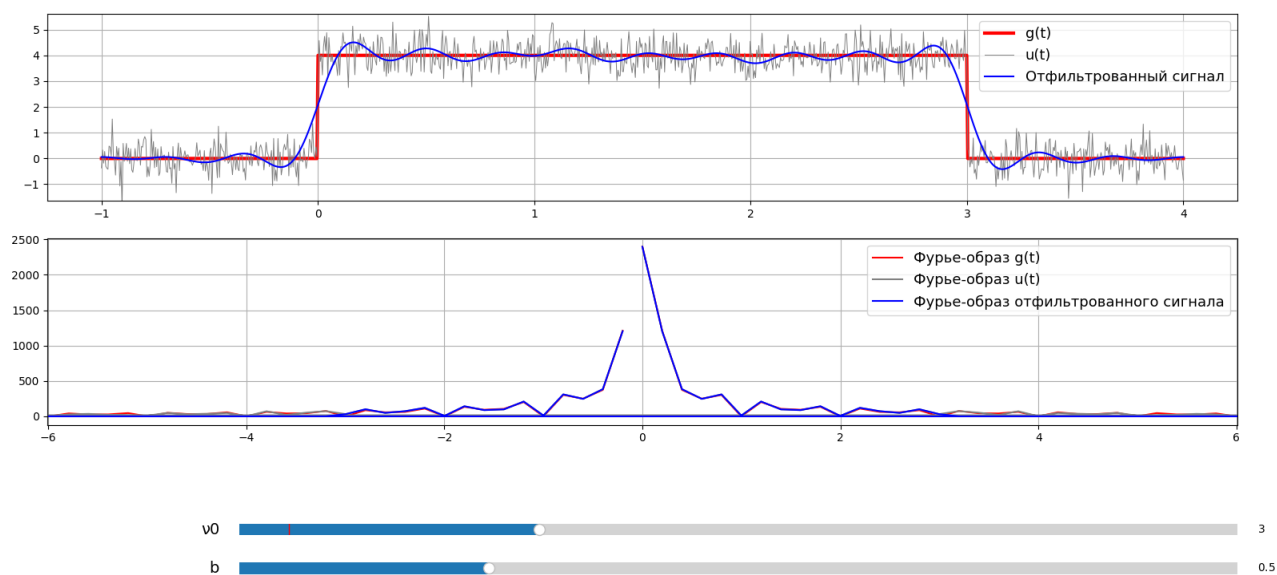


Рис. 8: Графики при $\nu_0 = 3$ и $b = 0.5$

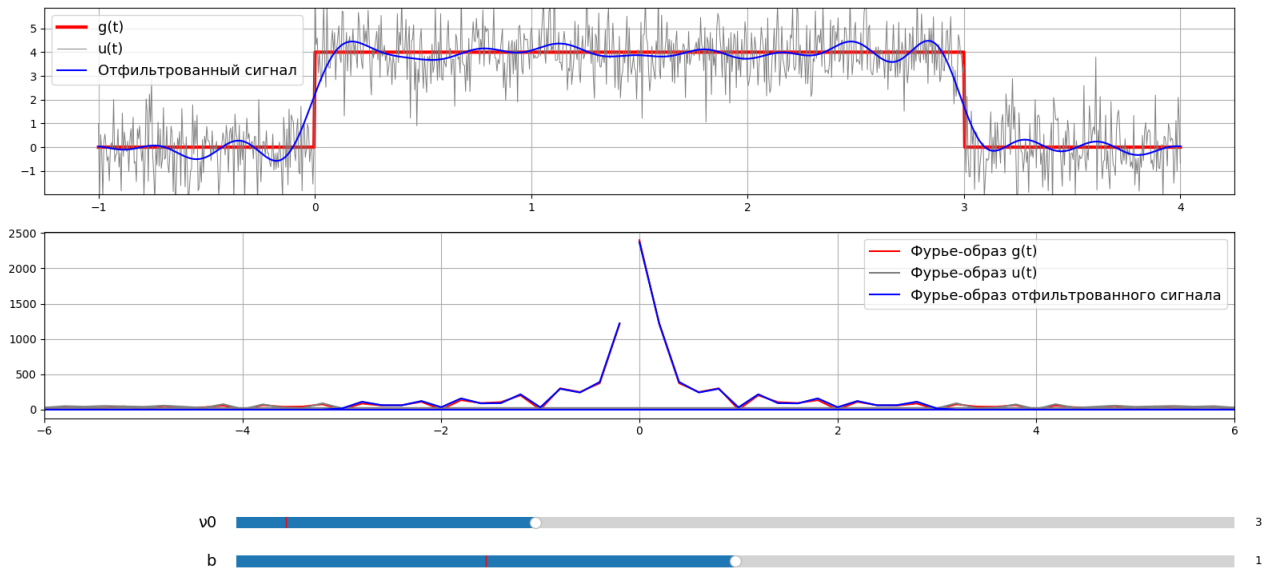


Рис. 9: Графики при $\nu_0 = 3$ и $b = 1$

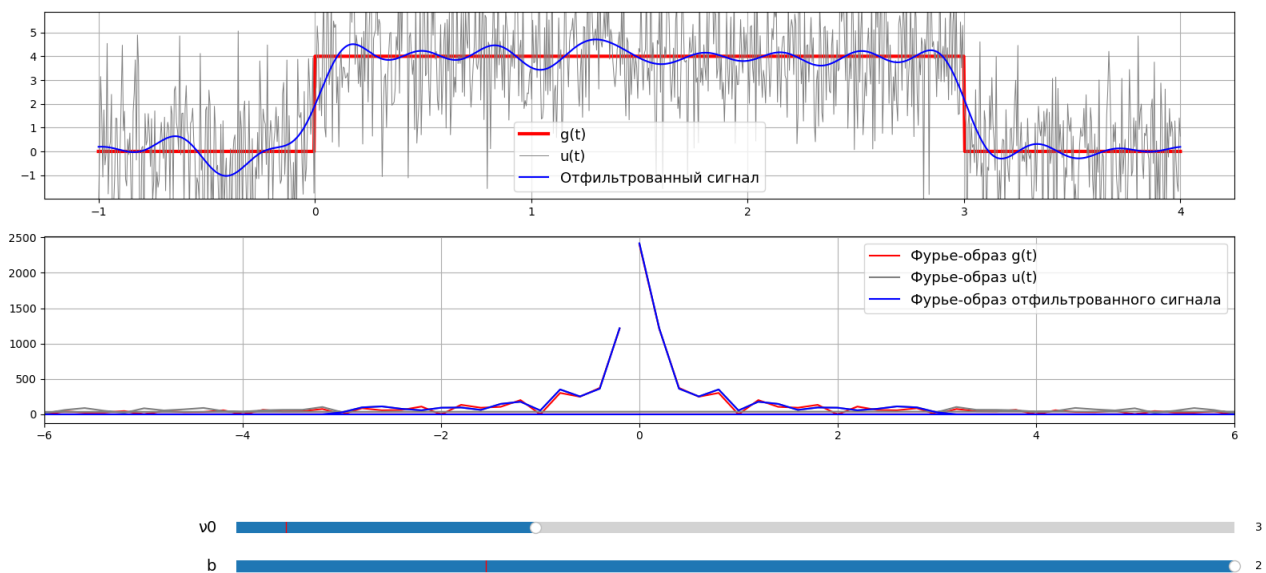


Рис. 10: Графики при $\nu_0 = 3$ и $b = 2$

1.2.5 Вывод

- При увеличении b увеличивается шум, а также растет амплитуда колебаний синего графика. То есть чем больше шум, тем больше помех в отфильтрованном сигнале. Идеальным графиком будет первый, поскольку шума совсем нет, соответственно мы фактически фильтруем не зашумленный сигнал, а идеальный.
- Спектр отфильтрованного сигнала близок к спектру $g(t)$, но остаются небольшие шумовые компоненты. Различия между модулями фурье-образов слабо заметны при моих параметрах b .

1.3 Убираем специфические частоты

1.3.1 Предподготовка

Для этого задания выберу параметры функций:

$$a = 4, t_0 = 0, t_1 = 3, c = 1, d = 2, b = 0.5$$

Значение частоты я выберу $\nu_0 = 0.5$. Стоит указать, что это только начальные значения и в ходе этого задания я буду их менять, чтобы посмотреть, как это отображается на графике.

У меня получатся следующие функции:

$$g(t) = \begin{cases} 4, & t \in [0, 3], \\ 0, & t \notin [0, 3], \end{cases}$$

$$u(t) = g(t) + 0.5\xi(t) + \sin(2t),$$

где $\xi(t) \sim U[-1, 1]$ — равномерное распределение на интервале $[-1, 1]$.

Отрисую график для выбранных значений:

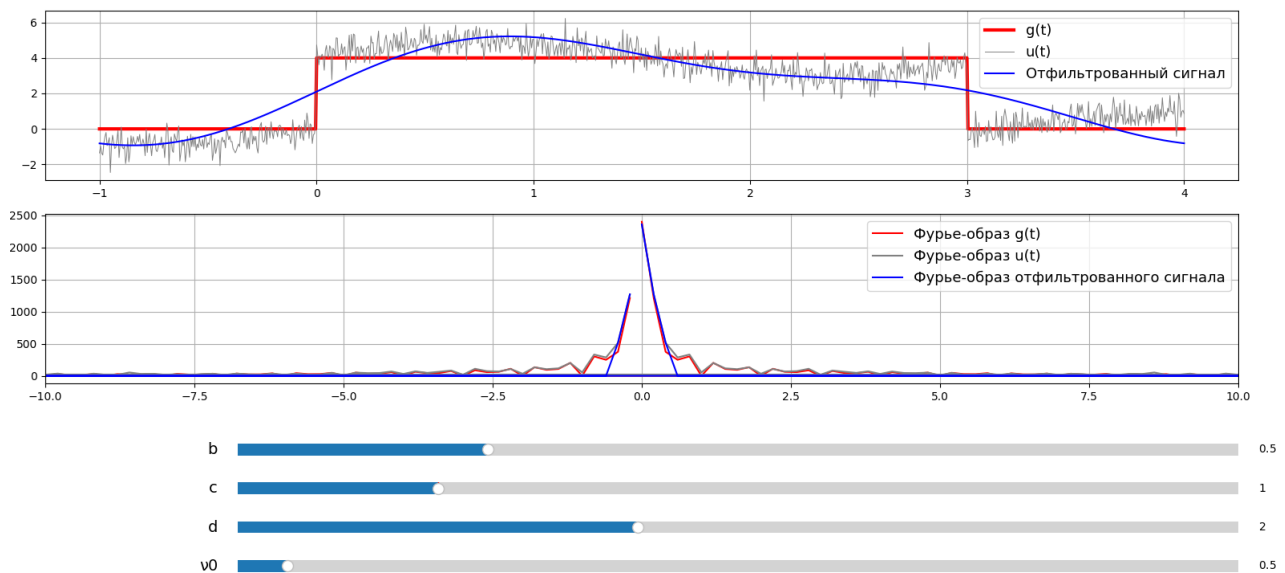


Рис. 11: Графики при $b = 0.5$, $c = 1$, $d = 1$ и $\nu_0 = 3$

В отличие от предыдущего пункта можно заметить, что шум пошел по синусоиде.

В этом задании нужно сделать совмещенный фильтр - он убирает не только низкие частоты, но и гармонические колебания. Алгоритм задания такой же, как и описывался в пункте 1.1.1.

Также отмечу, что, поскольку нужно исследовать довольно много меняющихся параметров в этом задании, я приведу несколько показательных случаев в следующих пунктах и отражу влияние компонент в выводе.

1.3.2 Случай, когда $b = 0$

Отдельно рассмотрю случай, когда $b = 0$. Вот несколько графиков

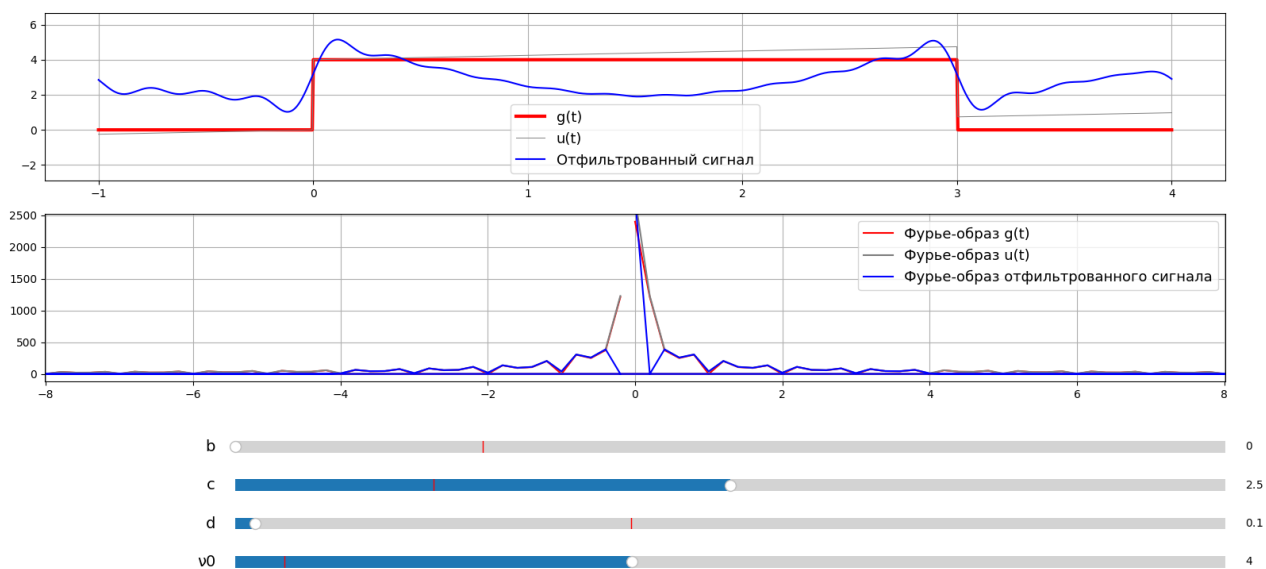


Рис. 12: Графики при $b = 0$, $c = 2.5$, $d = 0.1$ и $\nu_0 = 4$

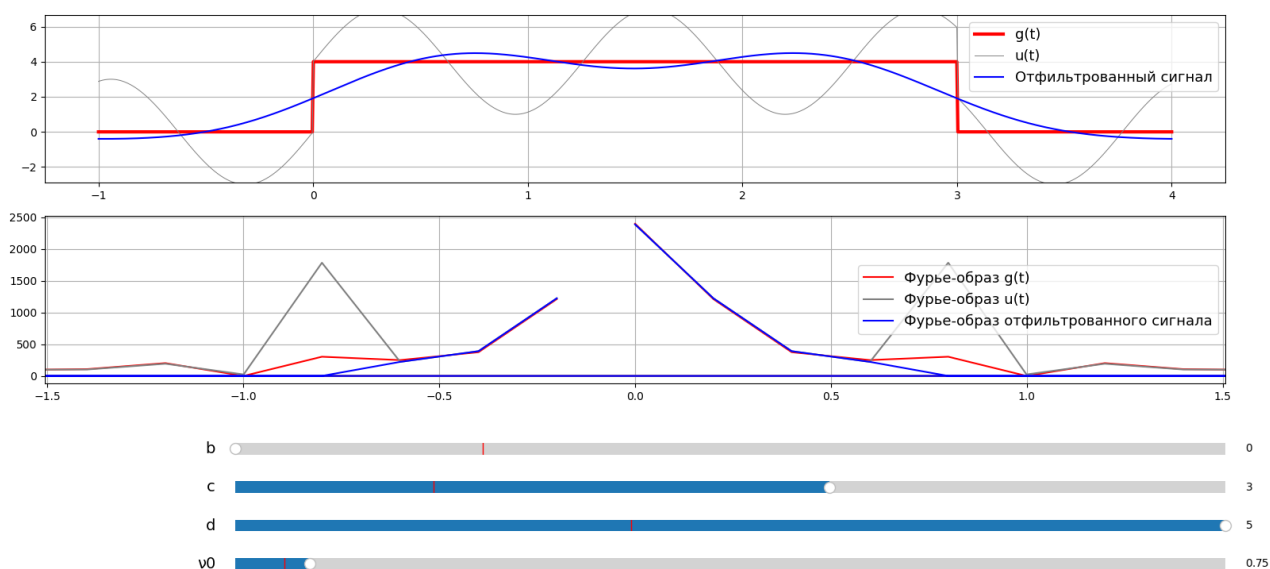


Рис. 13: Графики при $b = 0$, $c = 3$, $d = 5$ и $\nu_0 = 0.75$

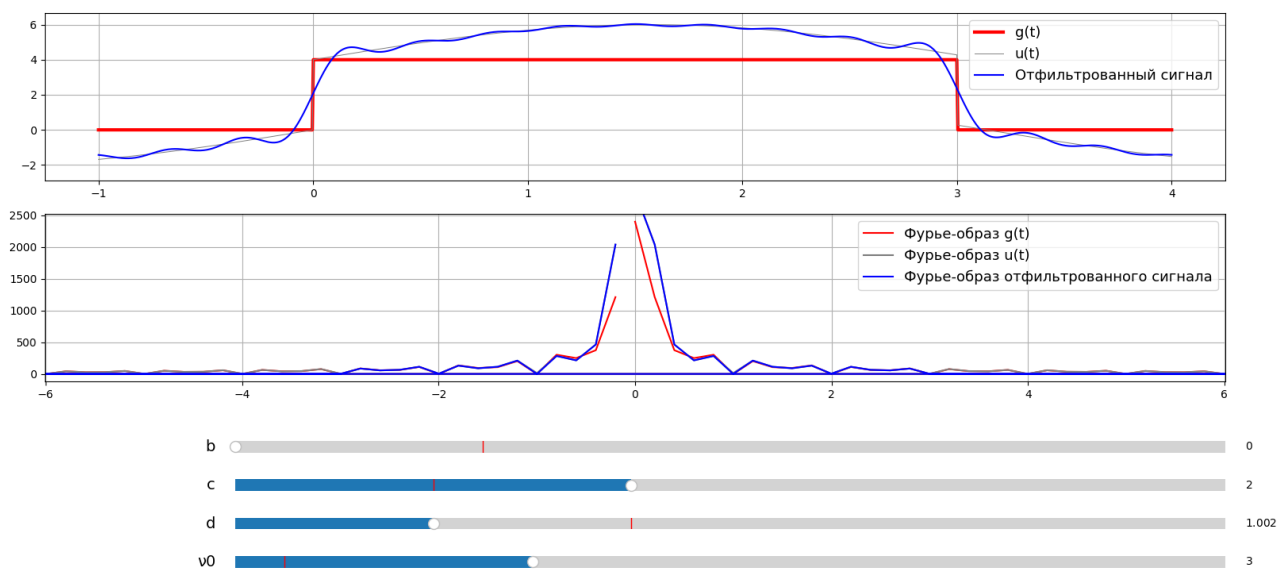


Рис. 14: Графики при $b = 0$, $c = 2$, $d \approx 1$ и $\nu_0 = 3$

1.3.3 Остальные случаи

Приведу графики, когда $b \neq 0$:

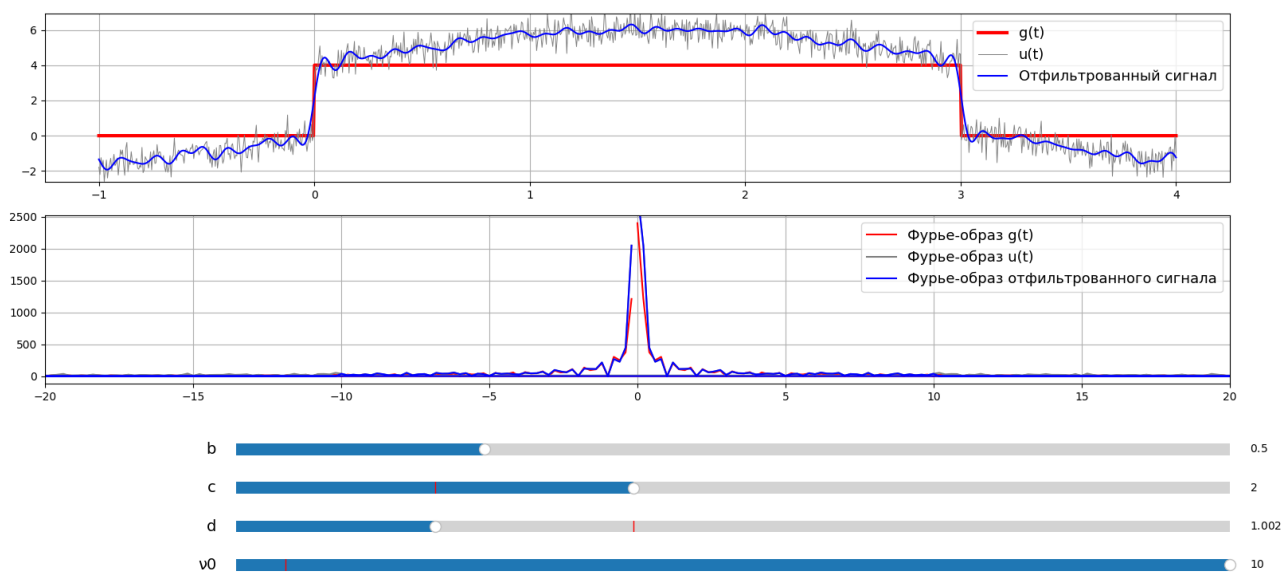


Рис. 15: Графики при $b = 2$, $c = 0.75$, $d \approx 5$ и $\nu_0 = 1$

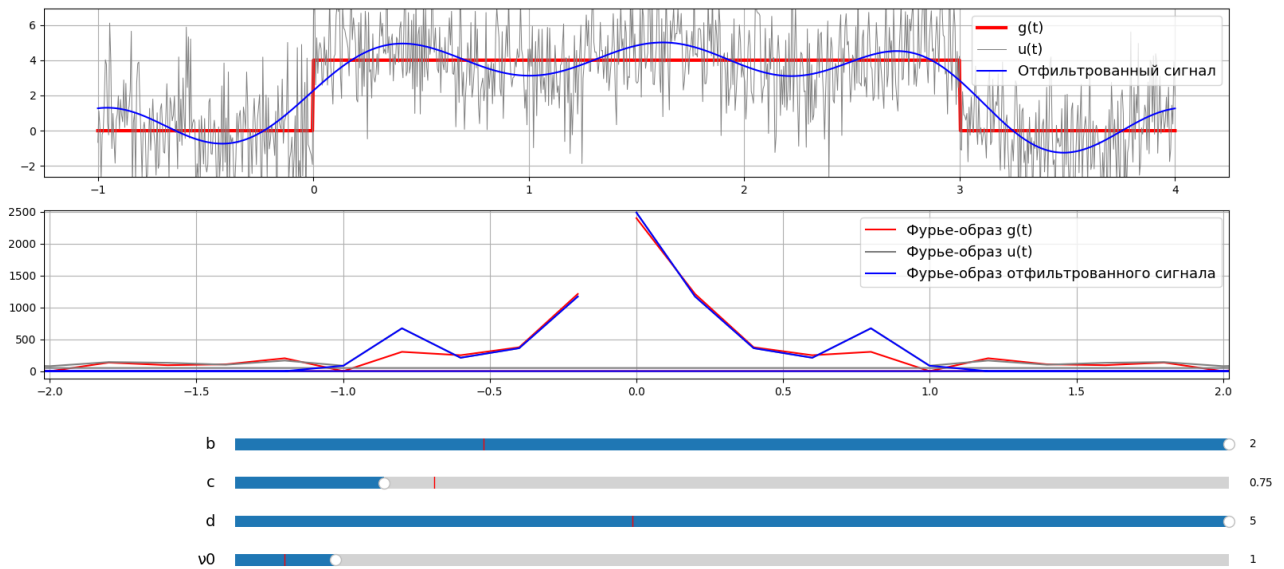


Рис. 16: Графики при $b = 0$, $c = 3$, $d = 5$ и $\nu_0 = 0.75$

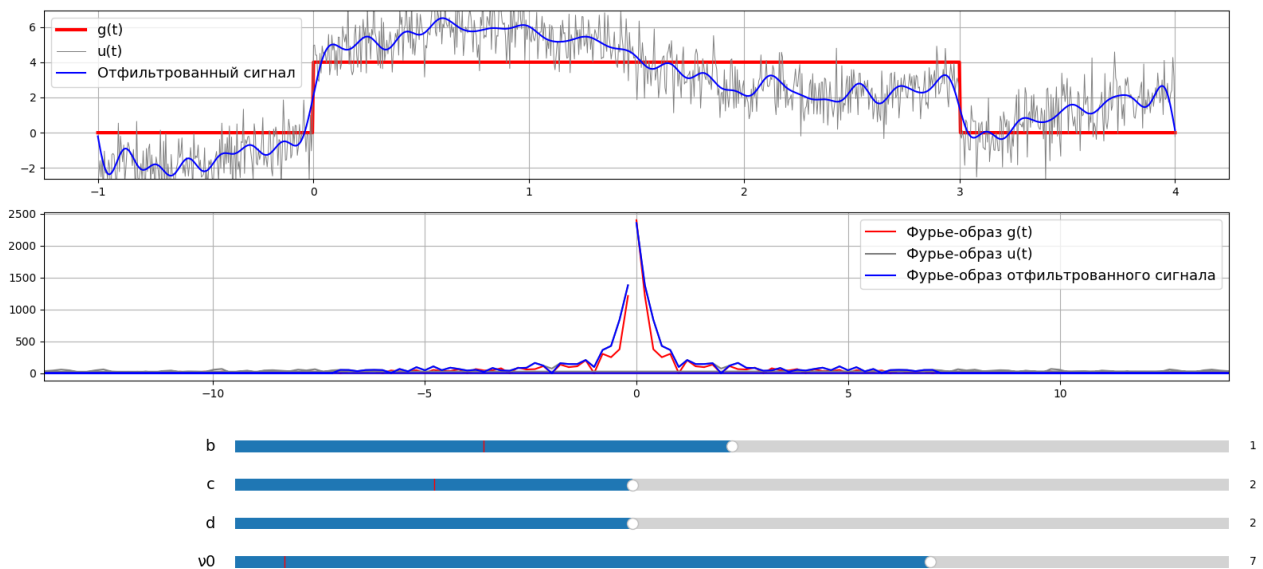


Рис. 17: Графики при $b = 1$, $c = 2$, $d = 2$ и $\nu_0 = 7$

1.3.4 Выводы по пунктам 1.3.2 и 1.3.3

Много раз поменяв параметры b , c , d и ν_0 , я пришла к следующим выводам:

- Параметр b : Чем больше значение b , тем больше шума накладывается на сигнал. Это связано с тем, что b определяет амплитуду шума.
- Параметр c : Увеличение c приводит к увеличению амплитуды колебаний зашумлённой функции. Также c влияет на количество колебаний шума, делая их более частыми или редкими.
- Параметр d : Увеличение d приводит к увеличению частоты гармонической составляющей шума, что делает колебания более частыми.

- Параметр ν_0 : Чем больше частота среза ν_0 , тем больше колебаний сохраняется в отфильтрованном сигнале. Это связано с тем, что фильтр пропускает больше высокочастотных компонент.

На графиках заметно, что совмещенный фильтр эффективно работает при присутствии колебаний в зашумленной функции.

1.4 Убираем низкие частоты?

1.4.1 Предподготовка

Для начала в этом задании я выберу такие функции:

$$g(t) = \begin{cases} 4, & t \in [0, 3], \\ 0, & t \notin [0, 3], \end{cases}$$

$$u(t) = g(t) + 0.5\xi(t) + \sin(2t),$$

где $\xi(t) \sim U[-1, 1]$ — равномерное распределение на интервале $[-1, 1]$.

И пропущу функцию $u(t)$ через фильтр, который обнуляет фурье-образ в окрестности точки $\nu = 0$

1.4.2 Графики

Изобразю график для выбранных ранее параметров:

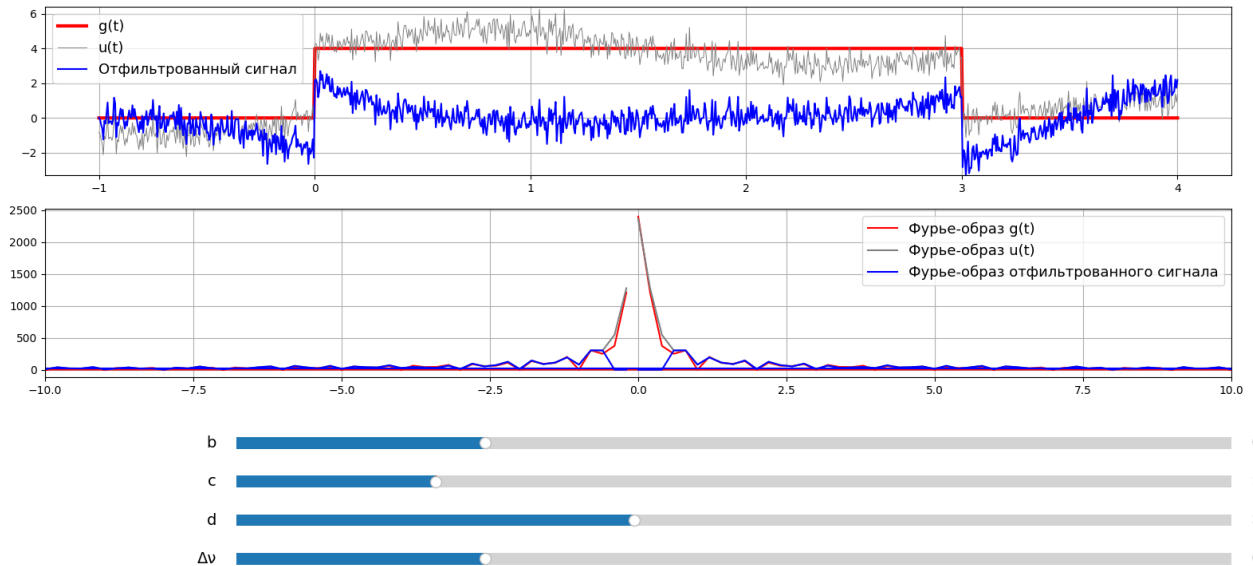


Рис. 18: Графики при $b = 0.5$, $c = 1$, $d = 2$ и $\Delta\nu = 0.5$

Далее буду менять окрестность точки ν - $\Delta\nu = \{0, 1\}$ для трех наборов параметров b , c , d , которые будут отражены внизу рисунка.

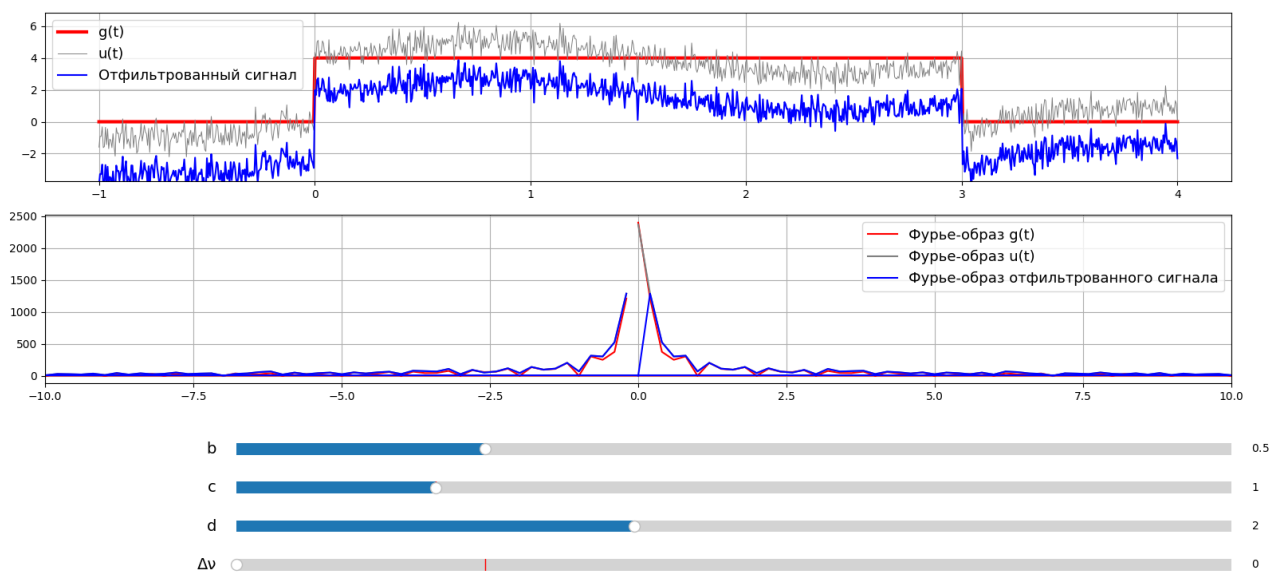


Рис. 19: Графики при $b = 0.5$, $c = 1$, $d = 2$ и $\Delta v = 0$

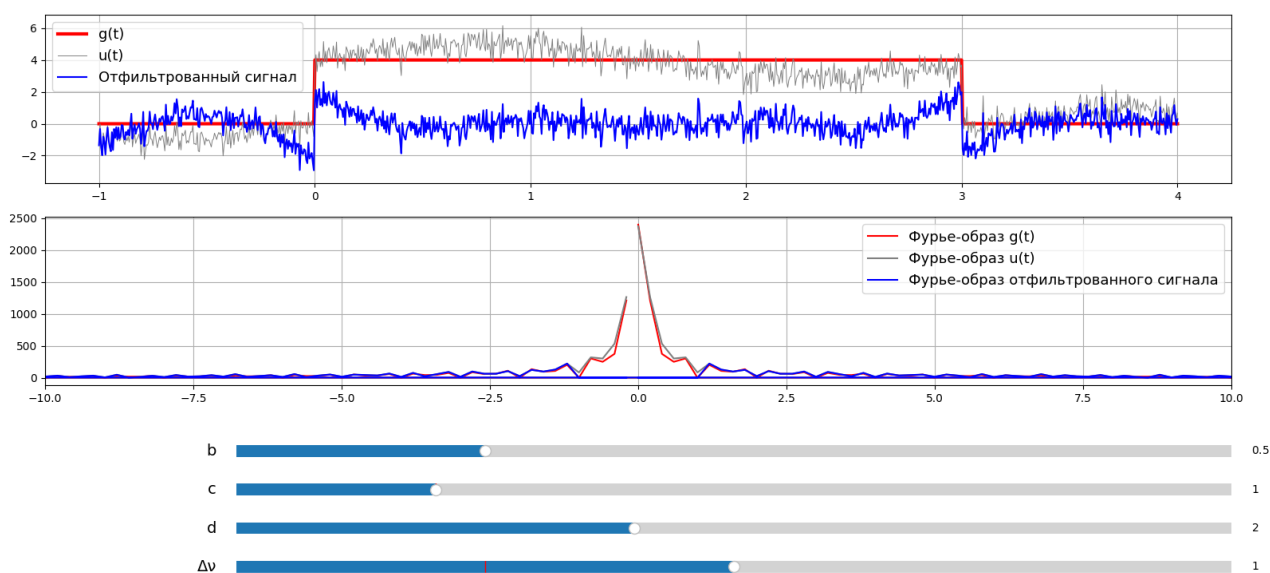


Рис. 20: Графики при $b = 0.5$, $c = 1$, $d = 2$ и $\Delta v = 1$

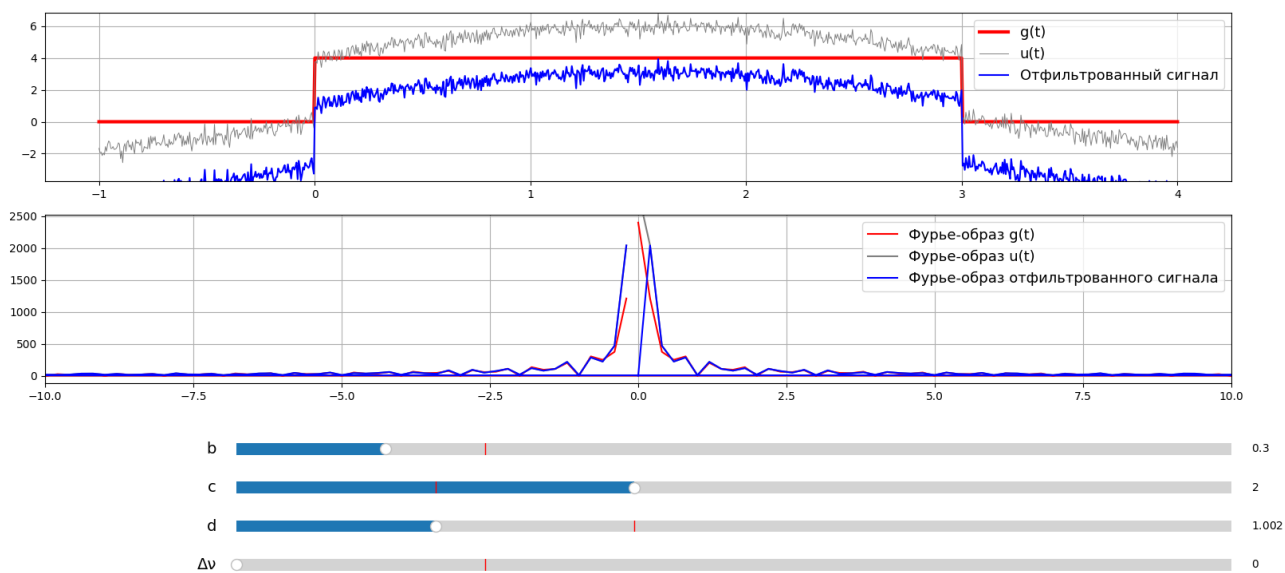


Рис. 21: Графики при $b = 0.3$, $c = 2$, $d \approx 1$ и $\Delta v = 0$

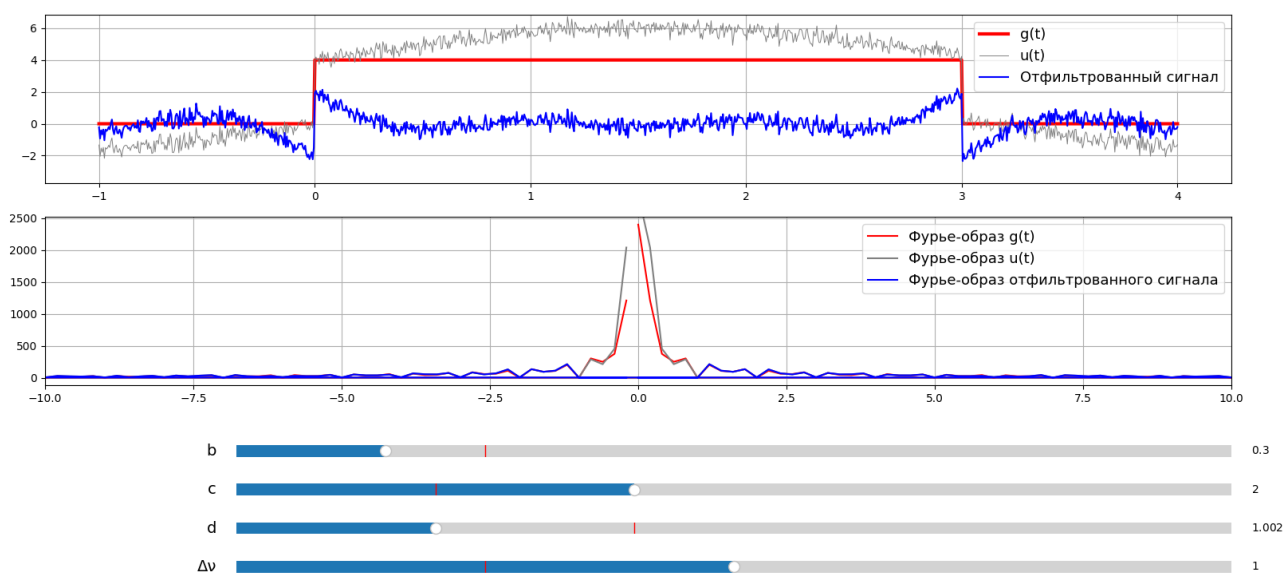


Рис. 22: Графики при $b = 0.3$, $c = 2$, $d \approx 1$ и $\Delta v = 1$

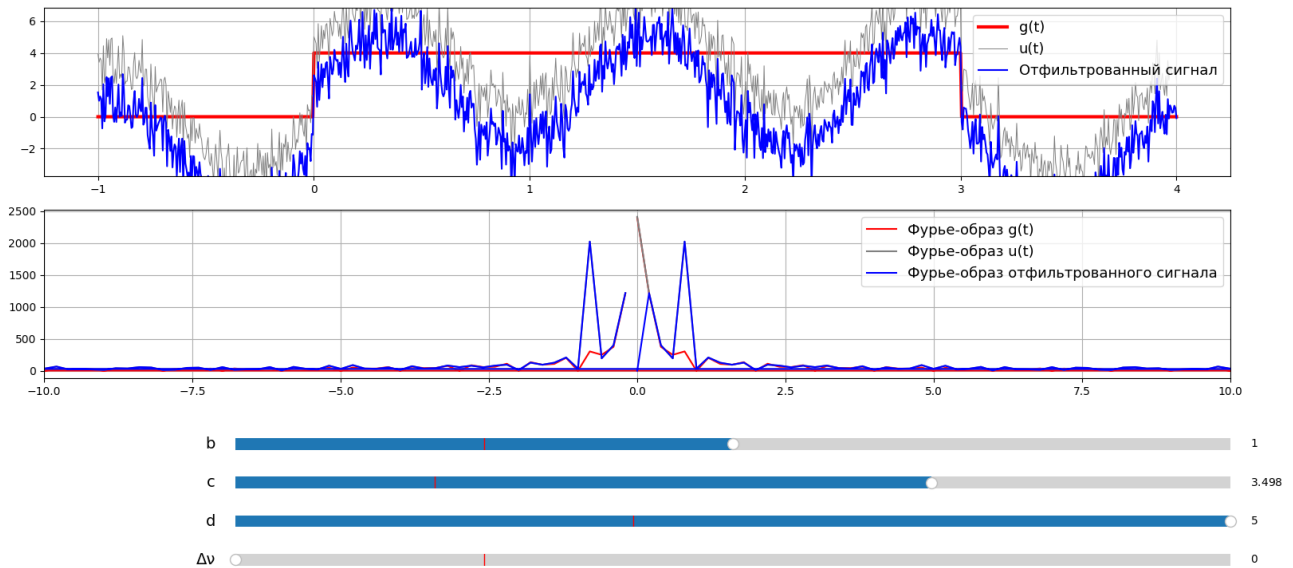


Рис. 23: Графики при $b = 1$, $c \approx 3.5$, $d = 5$ и $\Delta v = 0$

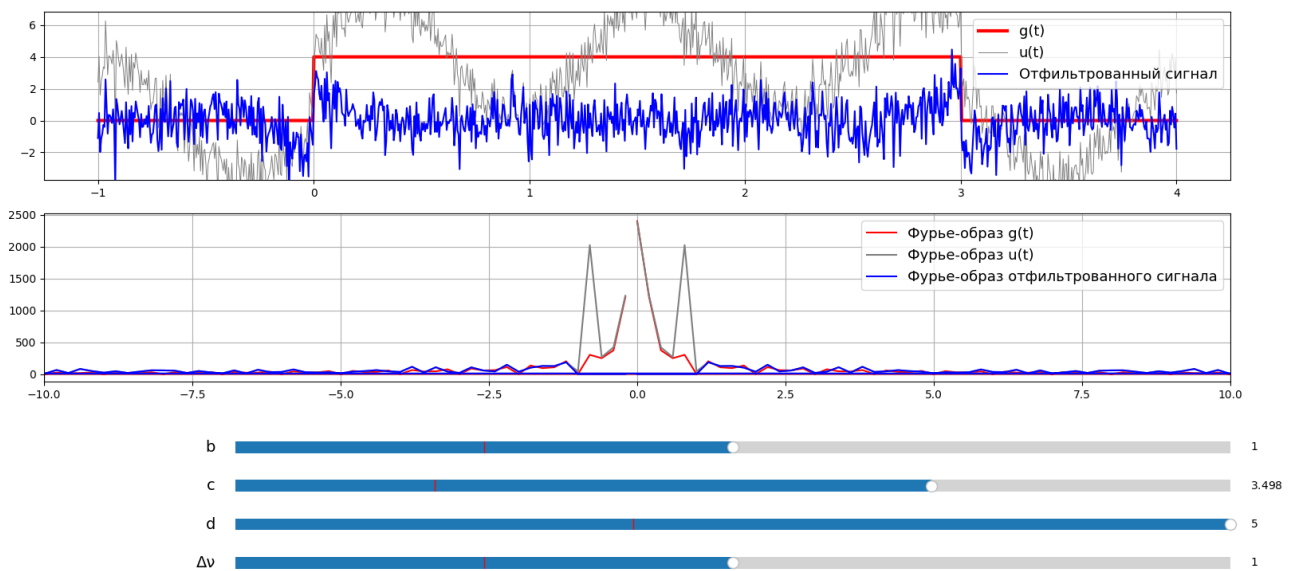


Рис. 24: Графики при $b = 1$, $c \approx 3.5$, $d = 5$ и $\Delta v = 1$

1.4.3 Выводы

- На графиках видно, что при разных значениях частот среза изменяется степень фильтрации и форма сигнала.
- Фильтр успешно подавляет низкочастотные компоненты сигнала, что видно по изменению формы отфильтрованного сигнала и его Фурье-образа. Низкочастотные компоненты вблизи ν практически отсутствуют после фильтрации.
- Однако данный фильтр не является идеальным решением. Подавление низких частот в сигнале может быть не всегда целесообразным, так как низкочастотные компоненты часто содержат полезную информацию. Кроме того, шум в высокочастотной области сигнала сохраняется, что может ухудшить качество обработки.

Таким образом, проведенный анализ подтверждает, что фильтр низких частот эффективно справляется с задачей подавления низкочастотных компонент сигнала, но его использование может быть ограничено из-за сохранения шума и возможного искажения полезной информации.

2 Task. Фильтрация звука

2.1 Краткое условие

Дан аудиофайл `MUNA.wav`, содержащий речь и шум. Необходимо отфильтровать сигнал так, чтобы осталась только речевая часть.

Ожидаемые результаты:

- Построены графики исходного и фильтрованного сигналов во времени;
- Построены графики модулей Фурье-образов этих сигналов;
- Описан выбранный диапазон фильтрации и причины выбора.

2.2 График исходного сигнала

Для начала построю график исходного сигнала. В питоне для этого к записи нужно применить `sf.read("MUNA.wav")`. И построить график зависимости амплитуды от времени. Посмотрим на получившийся график:

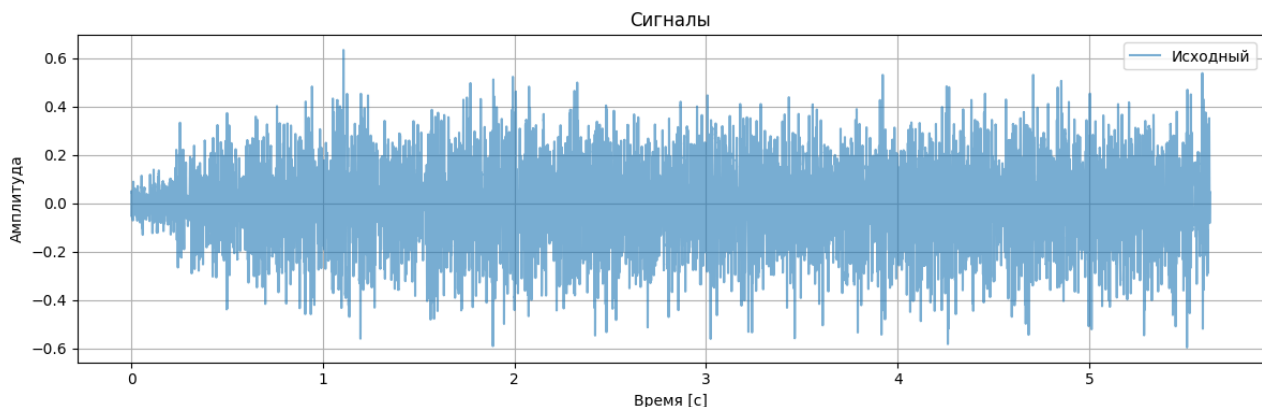


Рис. 25: График сигнала

Послушав запись, я выяснила, что шум существует на низкой частоте. Я отфильтровала низкие частоты и послушала получившуюся запись. Оказалось, что в записи есть еще немного высокого писка. Поэтому нужно отфильтровать и его. В качестве фильтра для этого задания я выбрала полосовой, который фильтрует сигнал в диапазоне. Размеры диапазона были подобраны самостоятельно.

Приступлю к написанию фильтра:

```
1 def bandpass_filter(data, lowcut, highcut, fs, order=5):
2     nyq = 0.5 * fs
3     low = lowcut / nyq
4     high = highcut / nyq
5     b, a = butter(order, [low, high], btype='band')
6     return filtfilt(b, a, data)
```

Листинг 3: Полосовой фильтр

Применю этот фильтр к сигналу. Оставлю частоты $\in [500, 3000]$ Гц:

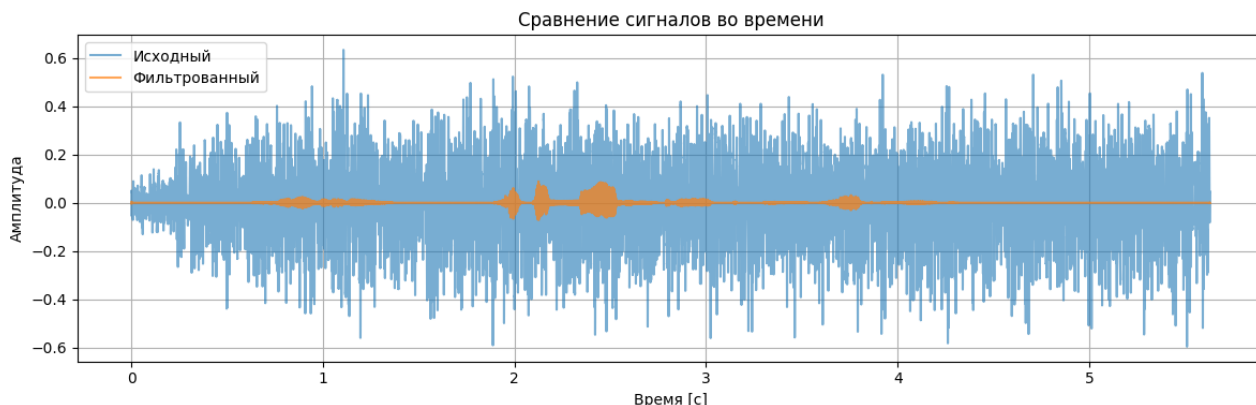


Рис. 26: График сигналов

Отфильтрованный сигнал можно послушать вот тут : [filtreted](#).

Можно заметить, что фильтр отлично сработал, много шума было удалено. Теперь посмотрим на графики модулей Фурье-образов исходного и фильтрованного сигналов:



Рис. 27: График спектра

На графике спектра видно, что после фильтрации осталась только часть сигнала в диапазоне от 500 до 3000 Гц. Низкие и высокие частоты были удалены — именно там находился основной шум.

Благодаря этому фильтр хорошо выделил голос. Основная энергия осталась в нужной полосе, а лишние шумы — исчезли. Это позволило сделать звук намного чище, при этом речь сохранилась.

3 Выводы

В этой работе я научилась фильтровать сигналы в частотной области. Сначала я работала с простыми функциями и шумами, потом — с реальной аудиозаписью. Я подобрала подходящий полосовой фильтр, который хорошо убрал лишние частоты и оставил только голос. Благодаря этому сигнал стал чище, а результат — лучше слышен. Всё это помогло мне лучше понять, как работает частотная фильтрация на практике. [GitHub](#).